

Custom Lab Proposal

1. Introduction

- Our project idea is a laser turret, which does color tracking in real time. We will be using FPGA computer vision along with microcontroller/pico control systems to make this possible. Essentially, how this works is the OV 7670 camera captures the raw pixels and sends it to the ICE Sugar Pro FPGA. The FPGA processes every pixel to detect the location of the user-specified color object. Finally, those targeted coordinates are sent to the pico, which runs a PID control loop for the servo Tourette, and this will mount the angle of the laser pointer to point directly at the target. Finally, the LCD displays a multi-panel view of what's going on in terms of the raw camera feed, the targeting scope of the object being identified, and hopefully a positioning map to display the target's movement. We're not sure if we should make this multi-panel or make this one screen and change it with a button; we will decide this later on as we work through it. However, essentially, this entire system is working in a closed feedback loop running at 30 frames per second.

2. Hardware

- ICESugar Pro FPGA
 - The purpose of the FPGA is for image processing; we will be using the FPGA feed to the BR image processing engine and also our display controller. It will capture the camera pixels and do the color space conversion along with the threshold on each pixel, so it can compute the centroid of the desired color. Hopefully, all this works right, and it drives the RGB LCD properly.
- RGB LCD display
 - We plan on making this multi panel, in the sense where we can have the one screen display 4 to 2 screens, or depending on the size of how the situation is looking, we can operate the multiple displays on the one screen with a button, because we want to show the raw camera feed, the binary threshold of the image detecting the targeted pixels, a positioning map to show the location relative to the laser along with a fading movement trail. And lastly, like a targeting box that overlays the object in the raw camera speed. We will figure out the details of how the setup of the display will look later on, but hopefully we will have all these views so it is clear for the user.
- Pico

- This is the system's control processor because it will receive the centroid coordinates from the FPGA using SPI, and then it will run a PID control loop on a timer callback to do the calculation for the servo angle adjustments. And ultimately, the output of the pico will be PWM signals to tilt the servos and angle it accurately to point to the target. We will handle the user input of when the laser should be on or off using GPIO interrupt for mood switching. This is mainly event-driven because the SPI will be receiving interrupts, and a timer for the callback PID Loop and the GPIO will be edge-triggered based on the user's button inputs. We will be sure that there will be no polling loops as we did in 120 B.
 - OV76700 Camera Module
 - This camera module will be capturing 320×240 videos for three frames per second in RGB 65 format. The output will be pixel data, which will be fed into the FPGA so it can read one pixel per clock edge directly, and this will be using I2C
 - Tilt Servo Bracket with SG90 Servos
 - Essentially, what this is is two servos with a mechanical mounting that will easily allow us to use the two servos for tilting and angling properly. So for the servos, there will be 21, which will be used for the horizontal X axis, and the vertical servos will help with the tilting and rotation. These servos will be controlled by the PWM coming from the pico. And we plan to mount the camera and the laser on the top of this. [PID = Proportional-Integral-Derivative --> used to move servos smoothly, ensuring it reaches target]
3. Functionality
- We will be using a real time color tracking pipeline. This means the FPGA will process the camera's pixels through several hardware pipelines, and we will do this by converting each pixel from RGB 565 to HSV color space and we will use this threshold to compare against the target color, and ultimately lower. The goal is to get XY coordinates. At the end of each frame, the centroid of the blob is going to be calculated, and this will give us the precise position of the frame of the object. We hope that this pipeline will run at wire speed with zero latency.
 - Another functionality would be the PID servo control. So the pico will receive the centroid coordinates from the FPGA using SPI interrupts and this will calculate the error between the target and the frame center. This closed feedback group is going to run in 30 iterations per second.
 - Another functionality is making our display panel a multi-panel. This means that we want to add four views to the RGB LCD in real time. This

means we wanna make sure we have rock camera feed, a binary threshold output, a targeting scope, highlighting with a box overlay, and lastly, a positioning map. The decision of making it composite into one screen, all that or making it one at a time controlled by a button will be decided later on, but hopefully, we will make it multi-panel, so there is more information provided in the display rather than only the raw output.

- We're also hitting one of the main requirements, which is driven architecture. All the pico software is interrupt and callback-driven. We are not using the same approach. We did in 120 B. Because SPI will be receiving interrupts and handling the central data, there is going to be a timer callback for the PID loop going out of fixed rate, and the GPO interrupt will handle the button pressing for remote, switching between idle and tracking. Hopefully, this will make sure we have efficient execution and no wasted CPU cycles.
- Lastly, a personal touch I would like to add to this is a nice web dashboard that has these displays working on a dashboard so it would be impressive in terms of having a nice user interface to clearly depict what is being targeted, by what position, and the respective output being displayed, etc.

** Below is a simplified summary if it helps :)

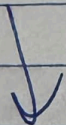
① Camera streams pixels



② FPGA processes them
+ finds target's pos



③ FPGA sends coord's to Pico
over SPI



④ PICO runs PID math +
adjusts servo



⑤ turret moves



⑥ Camera sees updated view

↳ ⑦ loop repeats 30 times/sec